

CODING CHALLENGE

SMART PACKAGE LOCKER MANAGEMENT SYSTEM



THE SCENARIO

Modern e-commerce platforms provide **self-service locker systems** where delivery agents can store packages for customers to pick up later.

Instead of delivering packages directly to homes, packages can be delivered to **secure locker stations** located in convenient public places such as malls, offices, or residential complexes.

HOW IT WORKS



Each locker station contains lockers of **different sizes**, allowing packages of different dimensions to be stored securely.



When a package is placed in a locker, the system generates a **pickup code** that is shared with the customer who can later use it to retrieve their package.



Locker system must efficiently manage **locker availability**, **package assignments**, and **package retrieval**.

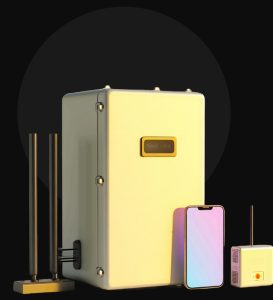


THE CHALLENGE

Build a **Package Locker Management System** that allows delivery agents to store packages and customers to retrieve them.

You may choose how the system is exposed (e.g., REST APIs, CLI, simple UI, or other interface).

SYSTEM OVERVIEW



The system manages

Lockers of different sizes.

Packages being delivered.

Customers receiving packages.

Pickup codes used for package retrieval.



When a delivery agent stores a package

The system finds a suitable and available locker.

Package is assigned to that locker.

A pickup code is generated.

The pickup code is assumed to be sent to the customer through an external notification system (e.g., SMS/email).



When the customer arrives to retrieve the package

The customer provides the locker ID and pickup code..

The system validates the request.

The locker opens and the package is retrieved.



Locker Sizes

Lockers come in different sizes:

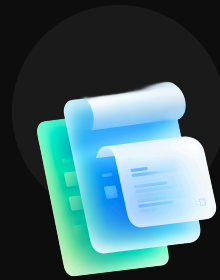
Small

Medium

Large

Packages have sizes and they must be placed in a locker that can hold it.

The system should always assign package to smallest available locker that can accomodate it.



Basic rules

A locker can store only **one package at a time**.

Once a package is retrieved, the locker becomes available again.

Each stored package must have a unique pickup code.

A pickup code is associated with a specific package and locker.

The system should be designed so it can be extended easily in the future.



Roles

Delivery Agent

Delivery agents interact with the system to store packages in lockers for customers by generating pickup codes

Customer

Customers interact with the system to retrieve their packages.

LEVEL 1: BASIC LOCKER AND PACKAGE STORAGE

Implement the basic locker allocation system.



Requirements

The system should support creating lockers of different sizes.

The system should allow viewing the list of lockers along with their current availability status.

Delivery agents should be able to store packages for customers in the locker system.

Delivery agents should be able to store packages for customers in the locker system.

When storing a package, the system should:

Find an available locker that can accommodate the package size

Prefer assigning the smallest available locker that fits the package.

If no suitable locker is available for the given package size, the system should return a message indicating that the package cannot be stored.

When a package is successfully stored, the system should generate a pickup code and provide the locker identifier where the package was stored.

The pickup code is assumed to be sent to the customer through an external notification system (such as SMS or email), which is outside the scope of this challenge.

LEVEL 2: PACKAGE RETRIEVAL AND LOCKER MANAGEMENT

Extend The System To Allow Customers To Retrieve Their Packages.



Requirements

Customers should be able to retrieve packages by providing the locker identifier and the pickup code they received.

The system should handle invalid scenarios properly.

When a valid pickup request is made:

- The corresponding locker should open
- The package should be removed from the locker
- The locker should become available again for future deliveries

LEVEL 3: EXTENDED STORAGE CHARGES

Extend the system to support storage charges for packages that remain in lockers for extended periods.

Packages are expected to be picked up within a reasonable time. If a package remains in the locker beyond a certain duration, additional storage charges may apply



Requirements

When a package is stored, the system should record the time or day when the package was placed in the locker.

The system should calculate storage charges based on how long the package stays in the locker.

For example, the storage fee may follow a tiered pricing rule such as charging X units /day for the first 5 days, 2X units /day for the next 5 days, and 3X units /day for any additional days where X is a fixed value.

Here the day is 24 hours from the time the package kept in the locker.

When a customer retrieves the package, the system should calculate the total storage charge based on the duration the package remained in the locker, and return this amount along with the pickup confirmation.

After retrieval, the locker should become available again for storing future packages.

LEVEL 4: HANDLING CONCURRENT REQUESTS (OPTIONAL)

Extend the system to correctly handle situations where multiple delivery agents try to store packages at the same time.

In real locker systems, several delivery agents may arrive at the same time and try to store packages. Since the number of lockers is limited, the system must ensure that lockers are assigned correctly without conflicts.



Requirements

The system should support cases where multiple storage requests happen at the same time.

When this happens, the system must ensure that:

- A locker is given to only one package at a time

- Two different requests should never receive the same locker.

- Locker availability should always remain correct and up to date.

If there are more requests than available lockers, only the available lockers should be assigned. The remaining requests should receive a message saying that no suitable locker is available

The system should continue to behave correctly even when many requests are made at the same time.

THANK YOU

We look forward to your solution.

All the best ✨

